

Setup guide: clone to running

Every command verified on a clean clone. No dataset, no GPU and no accounts required.

github.com/adityashuklaa/liver3d · python 3.10-3.12 · 45 tests · research prototype, not a medical device

Follow this top to bottom. Every command was run on a clean clone of this repository; the timings are from a laptop CPU with no GPU. You need **no dataset, no GPU and no accounts** to get the whole thing working.

Total: about 10 minutes, most of it waiting for `pip`.

0. Check what you have

```
python --version      # need 3.10, 3.11 or 3.12
git --version
```

If `python` is missing on Windows, install it from python.org and tick "**Add python.exe to PATH**" in the installer. On macOS use `python3` everywhere below.

Disk space: about 3 GB (PyTorch is 2 GB of it).

1. Clone the repository

```
git clone https://github.com/adityashuklaa/liver3d
cd liver3d
```

2. Create an isolated environment

Keeps these packages away from the rest of your machine.

```
python -m venv .venv
```

Activate it — this differs per shell:

SHELL	COMMAND
Windows PowerShell	<code>.env\Scripts\Activate.ps1</code>
Windows cmd.exe	<code>.env\Scripts\activate.bat</code>
Git Bash / macOS / Linux	<code>source .env/bin/activate</code>

Your prompt now starts with `(.env)` . If PowerShell refuses with an execution policy error, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` once, then activate again.

Every later step assumes this environment is active. Open a second terminal later? Activate it there too.

3. Install the dependencies

```
pip install -r requirements.txt
```

Takes 3–6 minutes; PyTorch is the slow part. This installs the CPU build, which is all the demo needs.

With an NVIDIA GPU, install the CUDA build *first*, then the rest:

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
```

Training picks the GPU up on its own — nothing to configure. Check with:

```
python -c "import torch; print(torch.cuda.is_available(), torch.__version__)"
```

4. Prove the install works

```
pytest -q
```

Expected: **45 passed** in roughly 10–30 seconds. These cover geometry round-trips, split-leakage detection, metric values, sliding-window blending, postprocessing, a full train → evaluate → infer cycle, and the API contract.

If this passes, everything below will work. If it fails, stop here — the failure message names the cause, and it is almost always a half-finished `pip install` .

5. Run the whole pipeline

```
python scripts/quickstart.py
```

Roughly 2–3 minutes on CPU. It generates 8 synthetic abdominal CT phantoms, locks a patient-level split, trains a small 3D U-Net, evaluates on the held-out case and segments one volume. Output ends with something like:

```
=== demo test-set Dice ===  
{ "n_cases": 1, "mean": 0.93, "median": 0.93 }
```

Your number will differ a little — training is stochastic. **These are synthetic phantoms: the score proves the pipeline is wired correctly, nothing about clinical accuracy.**

What it left behind:

outputs/demo/best.pt	trained checkpoint (weights + config together)
outputs/demo/config.yaml	exact configuration used
outputs/demo/env.json	python, torch, commit, hardware
outputs/demo/train_log.jsonl	loss and validation Dice per epoch
outputs/demo/curves.png	training curves
outputs/demo/run_summary.json	checkpoint checksum, model version
outputs/demo/eval_test/	per-case metrics CSV, summary, overlays, predictions
outputs/demo/inference/	mask, probability map, overlay, 3D surface, audit JSON
data/synthetic/	the generated phantoms

Faster smoke run if you are impatient: `python scripts/quickstart.py --cases 4 --epochs 3`.

6. Open the review workstation

Build the single-file viewer from the run you just did:

```
python scripts/export_viewer_data.py --checkpoint outputs/demo/best.pt --out outputs/viewer  
python scripts/build_viewer.py --data outputs/viewer --run outputs/demo --out viewer.html
```

The first command re-runs inference on three cases and exports slice images, masks, per-slice Dice and 3D surfaces. The second inlines all of it into one 1.8 MB HTML file — no server, no CDN, works offline.

Serve it (needed for the upload tab in step 7):

```
python -m http.server 8099
```

Open <http://127.0.0.1:8099/viewer.html>.

What to try:

- **Worklist** (left): click between studies; every panel follows.
- **Slice review**: drag the slider or scroll on the image; ← → step, space plays.
- **View modes**: Overlay, Prediction, Reference, **Difference** (red = the model went too wide, purple = liver it missed), CT only.
- **Dice by slice** (right): click the plot to jump to the worst slice.
- **Reader decision**: Accept / Correct / Reject writes an audit line.
- **3D surface**: drag to rotate, scroll to zoom.

Double-clicking `viewer.html` in a file manager also works for review — but the browser then blocks its calls to a local service, so uploads need the served URL.

7. Segment your own volume through the service

Open a **second terminal**, activate the environment again, and start the API:

PowerShell:

```
$env:LIVER3D_CHECKPOINT="outputs/demo/best.pt"
$env:LIVER3D_API_KEYS="demo-key:clinician"
$env:LIVER3D_CORS_ORIGINS="http://127.0.0.1:8099,null"
uvicorn src.api.service:app --port 8000
```

cmd.exe:

```
set LIVER3D_CHECKPOINT=outputs/demo/best.pt
set LIVER3D_API_KEYS=demo-key:clinician
set LIVER3D_CORS_ORIGINS=http://127.0.0.1:8099,null
uvicorn src.api.service:app --port 8000
```

macOS / Linux / Git Bash:

```
export LIVER3D_CHECKPOINT=outputs/demo/best.pt
export LIVER3D_API_KEYS=demo-key:clinician
export LIVER3D_CORS_ORIGINS=http://127.0.0.1:8099,null
uvicorn src.api.service:app --port 8000
```

Then in the viewer open **Upload study**, keep the endpoint `http://127.0.0.1:8000` and key `demo-key`, drop in a volume — try `data/synthetic/imagesTr/synth_002.nii.gz` — and press **Segment volume**.

You get a stated finding, for example:

```
Liver segmented: 232 ml as one connected structure. Proposed mask — a qualified reader must accept, correct or reject it.
```

with volume, slice count, runtime, job id and any warnings, and the study loads into the review pane. An empty result says so and is flagged for mandatory review; an ineligible file is rejected with the reason.

The API also works without the browser:

```
curl -X POST http://127.0.0.1:8000/v1/segmentations \
  -H "X-API-Key: demo-key" -F "file=@data/synthetic/imagesTr/synth_002.nii.gz"
curl http://127.0.0.1:8000/v1/segmentations/<job_id> -H "X-API-Key: demo-key"
```

Interactive docs: <http://127.0.0.1:8000/docs>

8. Command-line inference on a single file

No service, no browser:

```
python -m src.infer --checkpoint outputs/demo/best.pt \  
    --input data/synthetic/imagesTr/synth_003.nii.gz \  
    --out-dir outputs/inference
```

Writes the mask in the source geometry (opens aligned in 3D Slicer, ITK-SNAP or any NIfTI viewer), a probability map, an overlay PNG, a 3D surface and an audit JSON with the model version and checksum.

9. When you have real data (LiTS, CHAOS, hospital export)

```
# 1. put volumes and masks under data/lits as imagesTr/ + labelsTr/,  
#     or images/ + labels/, or flat volume-*.nii + segmentation-*.nii  
# 2. edit configs/baseline.yaml: data.root, data.dataset, and record the  
#     licence and approval in docs/data_card.md  
# 3. lock the split (patient level, seeded, checksummed)  
python scripts/build_manifest.py --root data/lits --out data_manifests/split_v1.csv \  
    --ratios 0.7 0.15 0.15 --seed 42  
  
# 4. train (GPU strongly recommended: hours, not minutes)  
python -m src.train --config configs/baseline.yaml  
  
# 5. tune on validation only  
python -m src.evaluate --checkpoint outputs/<run_id>/best.pt --split val  
  
# 6. once everything is frozen, open the test set exactly once  
python -m src.evaluate --checkpoint outputs/<run_id>/best.pt --split test
```

Then fill in `docs/validation_report_template.md`. Until that document is complete, the project's performance status is "pending verification" — do not quote a Dice number in a report before it is.

`docs/runbook.md` has the tuning order and the fixes for out-of-memory, stalled training and misaligned exports.

Troubleshooting

SYMPTOM	CAUSE AND FIX
<code>ModuleNotFoundError: No module named 'src'</code>	you are not in the repository root. <code>cd</code> into <code>liver3d</code> and re-run
<code>ModuleNotFoundError: No module named 'torch'</code>	the environment is not active, or <code>pip install</code> did not finish. Activate, re-run step 3
PowerShell blocks <code>Activate.ps1</code>	<code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code> , then activate
<code>pytest</code> says 0 tests	wrong directory again — run it from the repository root
Training is very slow	expected on CPU. Use <code>--cases 4 --epochs 3</code> , or a GPU build of torch
<code>CUDA out of memory</code>	lower <code>data.patch_size</code> to <code>[64,64,64]</code> and <code>training.batch_size</code> to 1 in the config, then delete <code>data/cache</code>
Port already in use	pick another: <code>python -m http.server 8100</code> , <code>uvicorn ... --port 8001</code> , and update the endpoint field in the Upload tab
Upload tab says the service is unreachable	the API terminal is not running, or the env vars were set in a different terminal than the one running uvicorn
Upload blocked over HTTPS	a page served over HTTPS cannot call a local HTTP service. Use <code>http://127.0.0.1:8099/viewer.html</code>
<code>viewer.html</code> shows old cases	rebuild it: <code>scripts/export_viewer_data.py</code> then <code>scripts/build_viewer.py</code>
First epoch is much slower than the rest	the preprocessing cache is being built once per configuration, under <code>data/cache/<hash>/</code>

Ground rules

- `data/`, `outputs/` and `*.pt` are gitignored. Never commit scans, masks or checkpoints trained on restricted data.
- Set `LIVER3D_API_KEYS` before exposing the service anywhere. Left unset, it prints a random development key and logs a warning.
- Do not put the service on a public network: no TLS, and keys travel in a header.
- This is a research prototype, not a cleared medical device. Every output is a proposed segmentation until a qualified reader accepts, corrects or rejects it.